PRAVAH

Complete Build Plan

Academic project-supervision management system: from partially built MVP to production-ready product

Stack	React, Redux, Node.js, Express, MongoDB / Mongoose
Roles	Student, Teacher, Admin
Scope	Foundation fixes, teacher module, admin and student completion, notifications, testing, security, deployment
Core milestone	Student submits a proposal, admin approves, teacher accepts, student uploads files, teacher gives feedback

Contents

- 1. Current status
- 2. Important problems to fix first
- 3. Recommended final product
- 4. Complete implementation roadmap (Phases 0 to 10)
- 5. Suggested implementation order (sprints)
- 6. Your immediate next step

1. Current status

PRAVAH is currently a **partially built MVP foundation**.

Already present

- React frontend with role-based routing
- Node.js/Express backend and MongoDB/Mongoose models
- Authentication: login, logout, registration, forgot password, reset password
- Three roles: Student, Teacher, Admin
- Admin CRUD for students and teachers
- Student proposal submission and file upload
- Supervisor listing and supervisor request creation
- Student feedback and notification models; project and deadline models
- Redux store structure and dashboard page structure
- File upload middleware and cloud/file service setup

The student backend is mainly in `studentController.js`, the admin backend in `adminController.js`, and routing is defined in `App.jsx`.

2. Important problems to fix first

Before adding new features, fix these foundation issues.

Backend issues

Issue	Detail
1. Teacher backend is missing	No complete teacher controller or router exists. The teacher dashboard, pending supervisor requests, assigned students and teacher files pages return empty fragments.
2. Several frontend pages are empty	Student dashboard, feedback and notifications; admin dashboard, supervisor assignment, deadlines and projects; teacher dashboard, requests, students and files contain little or no implementation.
3. Redux slices are incomplete	Teacher, request, notification, deadline and project slices have no working async actions or reducers, so most pages cannot yet talk to the backend.
4. Feedback field inconsistency	The project model defines `feedbacks` but the student controller uses `feedback`. This breaks dashboard feedback, retrieval, sorting and display. Use `feedbacks` everywhere.
5. Deadline model incorrectly exported	It exports a model named `Project` instead of a deadline model. Fix before building deadline APIs.
6. File service does not return the updated project	`addFilesToProject()` saves the project but does not return it, so the upload response may contain an undefined project.

Issue	Detail
7. Upload destination typo	The upload middleware uses `desination` instead of `destination`, which can stop files being stored in the intended directory.
8. Unfinished admin methods	Supervisor assignment and admin dashboard statistics are empty.
9. Missing notification APIs	Notifications are created internally, but there are no APIs to list, mark one read, mark all read, count unread or delete.
10. No test suite	The test script is still the default placeholder (`echo "Error: no test specified"`). Replace with real backend and frontend validation.

3. Recommended final product

PRAVAH should become an academic project-supervision management system. The complete lifecycle:

```
Admin creates users
        |
Student submits project proposal
        |
Admin reviews project
        |
Student requests a supervisor
        |
Teacher accepts or rejects request
        |
Supervisor is assigned
        |
Admin or supervisor sets milestones/deadlines
        |
Student uploads project files
        |
Teacher reviews files  ->  Teacher gives feedback
        |
Student updates project
        |
Teacher marks project completed
        |
Admin views reports and overall progress
```

This workflow is the central product. Every feature should support it.

4. Complete implementation roadmap

Phase 0: Define the product rules

Finalize business rules before coding further.

Project statuses

```
draft, submitted, under_review, approved, rejected,
in_progress, revision_required, completed, archived
```

Current statuses (`pending, approved, rejected, completed`) are too limited.

Supervisor request statuses

```
pending, accepted, rejected, cancelled
```

Who performs each action

Action	Responsible role
Create student / teacher	Admin
Submit project proposal	Student
Approve / reject proposal	Admin
Request supervisor	Student
Accept / reject supervisor request	Teacher
Manually assign supervisor	Admin
Set deadlines	Admin or assigned teacher
Upload files	Student
Review files and give feedback	Teacher
Mark project complete	Teacher / Admin
View reports	Admin

Both frontend and backend must follow this table consistently.

Phase 1: Stabilize the foundation

Complete this phase before building more UI.

Backend tasks

1. Fix the feedback/feedbacks inconsistency.
2. Fix the deadline model export.
3. Fix the file upload destination typo.
4. Make the file service return the updated project.
5. Add request validation (Joi, Zod or express-validator, used centrally).
6. Add pagination to users, projects, notifications and files.
7. Add consistent API response formats.
8. Add proper status transition validation.
9. Prevent duplicate supervisor requests and duplicate active projects.
10. Add database indexes for common queries.
11. Remove unused imports and unfinished functions.

Authentication hardening

1. Validate JWT expiry properly.

2. Configure secure cookies: `httpOnly`, `secure` in production, `sameSite`.
3. Rate-limit login, registration, forgot password and reset password.
4. Prevent users from changing their own role.
5. Prevent students from accessing other students' files or projects.
6. Prevent teachers from accessing unrelated projects.
7. Prevent deleted users from retaining active assignments.



Database integrity

- One active project per student
- One active supervisor per student
- Maximum students per teacher
- Valid student-teacher relationships
- Safe handling when deleting users with assigned projects
- Safe handling when deleting teachers who still supervise students

Phase 2: Complete admin functionality

Admin controls the rest of the system, so finish it next.

Admin dashboard

- Statistics: total students, teachers and projects; pending, approved, rejected and completed projects; unassigned students; teachers at capacity; upcoming deadlines
- Charts: projects by status, students by department, supervisor workload, monthly submissions, completed vs pending

Student and teacher management

- Create, edit, delete, search and filter by department
- Students: filter by project status, view details, project and assigned supervisor
- Teachers: view capacity, assigned student count and workload

Project management

- View all projects; search by title; filter by status, department and supervisor
- View full details, uploaded files and feedback history
- Approve proposal; reject with reason; assign supervisor; change status

Supervisor assignment: two flows

```
Student-request flow            Admin-assignment flow
--------------------            ---------------------
Student requests Teacher        Admin selects Student
Teacher accepts request         Admin selects Teacher
System assigns Teacher          System checks teacher capacity
System updates project          System assigns teacher
System updates teacher's        System updates both users
  assignedStudents              System sends notifications
System sends notifications
```

Assignment should be transactional, or carefully rolled back if one database update fails.

Deadline management

- Create global and project-specific deadlines
- Edit, delete, and mark active/inactive
- View upcoming deadlines and filter overdue ones

Phase 3: Build the complete teacher module

The most important immediate feature area.

Teacher backend

Create `teacherController.js` and `teacherRoutes.js`. Recommended endpoints:

```
GET  /api/v1/teacher/dashboard
GET  /api/v1/teacher/requests
PUT  /api/v1/teacher/requests/:id/accept
PUT  /api/v1/teacher/requests/:id/reject
GET  /api/v1/teacher/students
GET  /api/v1/teacher/students/:id
GET  /api/v1/teacher/projects
GET  /api/v1/teacher/projects/:id
GET  /api/v1/teacher/projects/:id/files
GET  /api/v1/teacher/projects/:projectId/files/:fileId
POST /api/v1/teacher/projects/:id/feedback
PUT  /api/v1/teacher/projects/:id/status
GET  /api/v1/teacher/deadlines
```

Teacher dashboard

- Assigned student count and maximum capacity
- Pending supervisor requests
- Active projects and projects requiring review
- Upcoming deadlines
- Recently uploaded files and recently submitted proposals

Pending requests page

- View student name, department, proposed project, message and request date
- Accept, or reject with a reason; filter pending / accepted / rejected

When accepting a request, the backend must:

1. Confirm teacher capacity.
2. Confirm the student has no supervisor yet.
3. Confirm the request is still pending.
4. Assign teacher to student and to project.
5. Update request status.
6. Notify the student.

Assigned students and files pages

- Students: name, email, department, project title and status, last upload, last feedback date, next deadline, progress state, links to details
- Files: view, download, upload date, file type, add feedback, mark reviewed, request revision

Feedback system

- Title, message, type, created date, teacher identity, read/unread state, revision-requested flag
- Student sees the complete feedback history

Phase 4: Complete the student module

Dashboard

- Current project, status, supervisor, submission date, latest feedback
- Upcoming deadlines, recent notifications, uploaded files, revision requirements, completion progress

Recommended progress steps:

```
Proposal submitted -> Supervisor assigned -> Project approved ->
Work in progress -> Files submitted -> Feedback received ->
Final revision -> Completed
```

Pages

- **Proposal:** title/description validation, character counter, save draft, submit, view rejection reason, resubmit, disable when an active project exists
- **Supervisor:** available teachers with department, expertise and capacity; request status and message; cancel request; assigned supervisor details; no repeated requests to the same teacher
- **Files:** drag-and-drop, size display, type validation, upload progress, delete before review, download, history, version number
- **Feedback:** timeline, type badges, supervisor name, date, revision indicator, mark as read, clarification reply
- **Notifications:** read/unread and type filters, mark one/all read, navigate to related page, delete

Phase 5: Build the notification system

Notification triggers

- Proposal submitted, approved, rejected
- Supervisor request received, accepted, rejected; supervisor assigned by admin
- File uploaded; feedback added; revision requested
- Deadline approaching or overdue; project marked completed

Notification API

```
GET    /api/v1/notifications
GET    /api/v1/notifications/unread-count
PUT    /api/v1/notifications/:id/read
PUT    /api/v1/notifications/read-all
DELETE /api/v1/notifications/:id
```

Real-time delivery (Socket.IO or server-sent events) comes later, only after the normal APIs work correctly.

Phase 6: Improve the project model

Recommended project fields

```
student, supervisor, title, description, department, status,
rejectionReason, revisionReason, submittedAt, approvedAt,
completedAt, lastActivityAt, files, feedbacks, deadlines,
progress, createdAt, updatedAt
```

File subdocument

```
originalName, storedName, fileUrl, fileType, fileSize,
version, uploadedBy, uploadedAt, reviewedAt, reviewStatus
```

Feedback subdocument

```
supervisorId, title, message, type,
requiresRevision, isRead, createdAt
```

Status transition rules

Do not allow arbitrary status changes from the frontend. The backend must enforce transitions such as:

```
submitted          -> under_review
under_review       -> approved
under_review       -> rejected
approved           -> in_progress
in_progress        -> revision_required
revision_required -> in_progress
in_progress        -> completed
```

Phase 7: Frontend completion and design system

Build after backend contracts are stable.

Shared UI components

Button, Input, Select, Modal, Table, Pagination, Search bar, Filter panel, Status badge, Loading / Empty / Error states, Confirmation dialog, File card, Notification item, Project status timeline.

Every API page must handle

Loading; success with data; success with empty data; validation error; unauthorized; forbidden; server error; retry. Never leave a blank page when an API fails.

Layout improvements

- Responsive sidebar and mobile navigation
- Consistent dashboard header, breadcrumbs, profile menu
- Logout confirmation
- Accessible forms, keyboard navigation, visible focus states

Phase 8: Testing

Backend unit tests

Registration, login (valid and invalid), password reset, role authorization, project creation, duplicate project prevention, supervisor request creation / acceptance / rejection, teacher capacity limits, file and feedback authorization, deadline creation, notification creation.

Backend integration workflows

Workflow	Steps
Student project	Login, submit proposal, admin approves, request supervisor, teacher accepts, upload file, teacher gives feedback, student views feedback, teacher completes project
Rejection	Submit proposal, admin rejects, student sees reason, student resubmits
Capacity	Teacher reaches max capacity, student cannot request teacher, admin cannot assign another student

Frontend tests

Protected routes, role redirects, login form, proposal form, file upload, supervisor request, teacher acceptance, admin filters, notification read state, empty and error states.

Manual acceptance testing

Create `admin@test.com`, `teacher@test.com` and `student@test.com`, then run the complete workflow in the browser.

Phase 9: Security and production hardening

- Strong production JWT secret; never commit `.env`; add `.env.example`
- Hash all passwords; never return password fields
- Secure cookies and rate limiting

- Validate uploaded file type (and actual contents where possible); restrict file download authorization
- Maximum request body size; security headers with Helmet
- Prevent NoSQL injection; sanitize user input
- Do not expose stack traces in production
- Add audit logs for admin actions

File storage

For production, do not rely only on local disk. Use Cloudinary, Amazon S3, Cloudflare R2 or similar. Store only metadata in MongoDB: file URL, name, size, type, owner, project and upload date.

Phase 10: Deployment

Layer	Options
Frontend	Vercel, Netlify, Cloudflare Pages
Backend	Render, Railway, Fly.io, AWS, DigitalOcean
Database	MongoDB Atlas
File storage	Cloudinary or S3-compatible storage

Production environment variables

```
Backend:
NODE_ENV, PORT, MONGO_URI, JWT_SECRET, JWT_EXPIRE, FRONTEND_URL,
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD,
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET

Frontend:
VITE_API_URL
```

Deployment checklist

- Frontend builds successfully; backend runs with production environment
- CORS, HTTPS, database IP access, email and file storage configured
- Test login, file upload, download, password reset, all roles and mobile layout
- Configure error logging and backups

5. Suggested implementation order

Sprint	Focus	Deliverables
1	Stabilization	Fix model and service bugs, upload handling, feedback naming, deadline model; add validation and basic API error handling
2	Teacher backend	Controller and routes, request accept/reject, assigned students, project access, feedback APIs
3	Teacher frontend	Dashboard, pending requests, assigned students, files page, feedback submission
4	Admin completion	Dashboard, proposal approval/rejection, supervisor assignment, project and deadline management
5	Student completion	Dashboard, feedback page, notifications page, file history, progress timeline
6	Notifications	APIs, read/unread state, triggers, deadline reminders
7	Testing and security	Backend and frontend tests, authorization tests, file security, rate limits, error handling
8	Deployment	MongoDB Atlas, cloud file storage, backend and frontend deploy, production testing, documentation

6. Your immediate next step

Do not start with chat, AI, mobile apps or advanced analytics yet. Follow this sequence:

1. Fix the project model and file upload bugs.
2. Implement the complete teacher backend.
3. Connect teacher Redux actions.
4. Build the teacher pages.
5. Implement admin proposal approval and supervisor assignment.
6. Complete the student dashboard and feedback flow.
7. Add deadlines and notifications.
8. Add tests.
9. Deploy.

Most important milestone

A student can submit a proposal, an admin can approve it, a teacher can accept the student, the student can upload files, and the teacher can give feedback.

Once this workflow works reliably end to end, PRAVAH has a usable core product. Everything else should be built around improving, securing and scaling it.